Trackalways Gravity — Engineering Specification

Version: 1.0.0
Platform: Family Safety & Real-Time Location Tracking
Author: Rodney Otieno
Date: June 2026
Document Type: Full Engineering Reference

Table of Contents

---

1. System Overview

Trackalways Gravity is a multi-platform family safety application that provides:

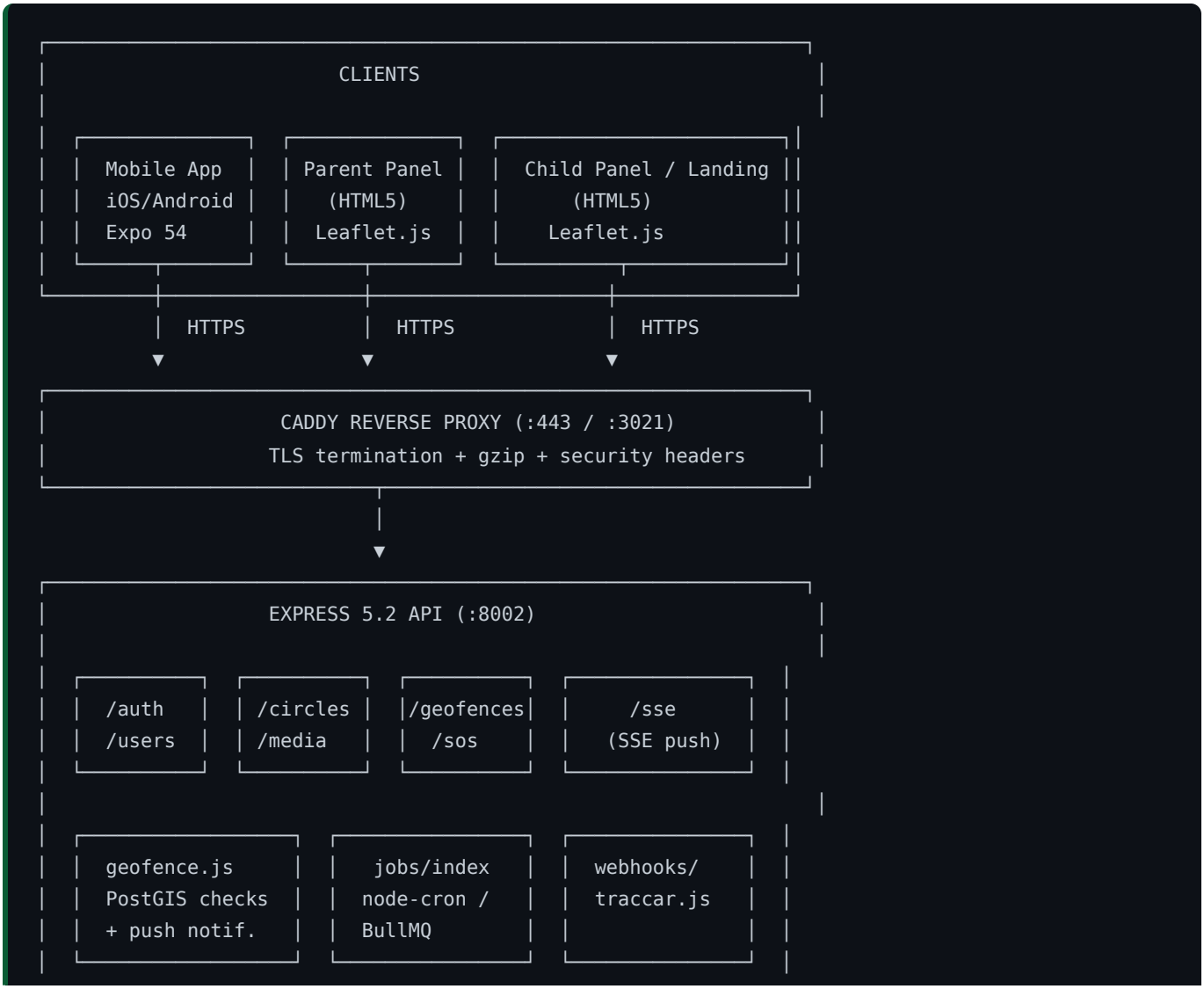
- **Real-time GPS tracking** of family members on an interactive map
- **Geofence (safe zone) management** with automatic entry/exit notifications
- **SOS emergency alerts** from child to all family members instantly
- **Family circles** — invite-based group membership with admin controls
- **Push notifications** for all critical events (SOS, geofence breach, low battery)
- **Cross-platform support** — iOS, Android (React Native / Expo), and Web (responsive HTML)

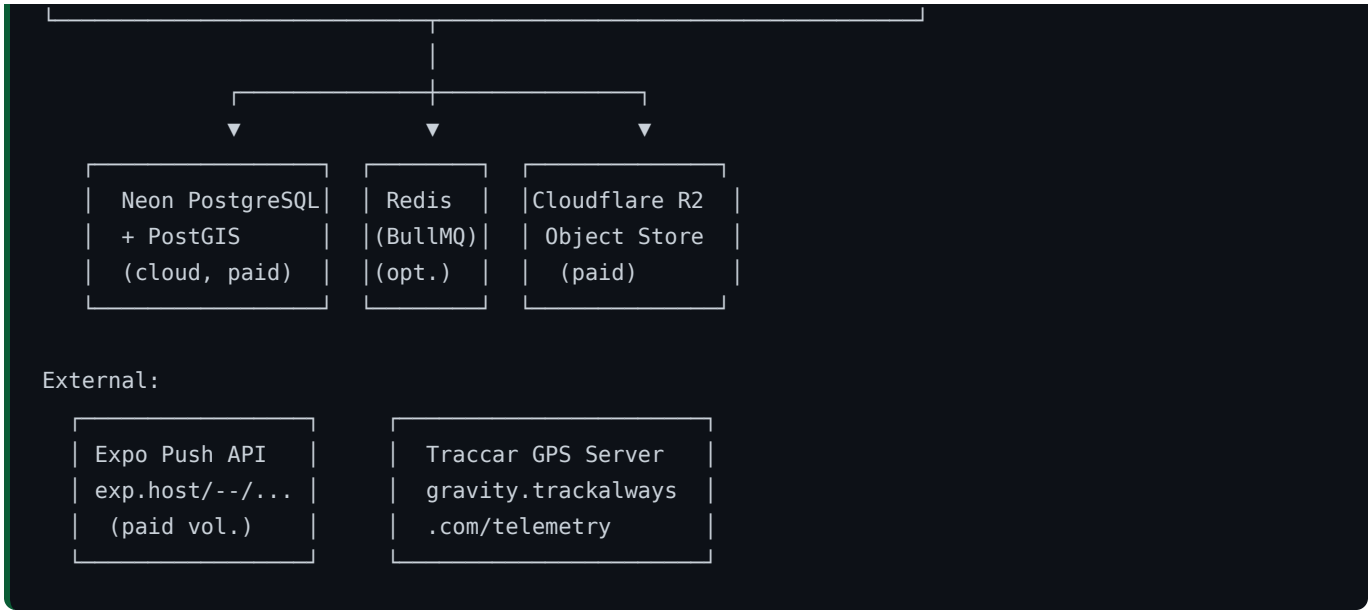
Components

| Component | Technology | Purpose | |-----------|-----------|---------| | Backend API | Node.js 20 + Express 5.2 | REST API, SSE, webhooks | | Database | PostgreSQL 16 + PostGIS 3 | All data, spatial queries | | Mobile App | Expo 54 + React Native 0.81 | iOS + Android app | | Web Landing | Vanilla HTML/CSS/JS + Leaflet.js | Marketing + web dashboards | | Reverse Proxy | Caddy 2 | TLS, routing | | Object Storage | Cloudflare R2 | Avatars, circle icons | | GPS Telemetry | Traccar (self-hosted) | Hardware GPS device ingestion |

---

2. Architecture Diagram





---

3. Technology Stack

Backend

| Package | Version | Purpose | License | |---|---|---|---| | Node.js | 20 LTS | JavaScript runtime | MIT (FREE) | | Express | 5.2.x | HTTP framework (latest stable) | MIT (FREE) | | pg | 8.12.x | PostgreSQL client (node-postgres) | MIT (FREE) | | @neondatabase/serverless | 1.1.x | Neon serverless HTTP driver | MIT (FREE) | | jsonwebtoken | 9.0.x | JWT sign/verify (HS256) | MIT (FREE) | | bcryptjs | 2.4.x | Password hashing (bcrypt, 10 rounds) | MIT (FREE) | | helmet | 7.1.x | HTTP security headers | MIT (FREE) | | cors | 2.8.x | Cross-Origin Resource Sharing | MIT (FREE) | | express-rate-limit | 7.3.x | API rate limiting | MIT (FREE) | | morgan | 1.10.x | HTTP request logger | MIT (FREE) | | zod | 3.23.x | Runtime schema validation | MIT (FREE) | | uuid | 10.0.x | UUID v4 generation | MIT (FREE) | | dotenv | 16.4.x | .env file loader | MIT (FREE) | | @aws-sdk/client-s3 | 3.600.x | S3-compatible R2 client | Apache-2 (FREE) | | @aws-sdk/s3-request-presigner | 3.600.x | Presigned URL generation | Apache-2 (FREE) | | bullmq | 5.12.x | Redis-backed job queue | MIT (FREE) | | ioredis | 5.4.x | Redis client for BullMQ | MIT (FREE) | | node-cron | 3.0.x | Fallback cron scheduler | MIT (FREE) | | ws | 8.21.x | WebSocket (used for SSE fallback) | MIT (FREE) | | nodemon | 3.1.x | Dev auto-restart | MIT (FREE, devDep) |

Mobile (Expo / React Native)

| Package | Version | Purpose | License | |---|---|---|---| | Expo | 54.0.x | Mobile build toolchain + OTA updates | MIT (FREE) | | React Native | 0.81.x | Cross-platform mobile UI | MIT (FREE) | | expo-router | 4.0.x | File-based routing (like Next.js) | MIT (FREE) | | @react-navigation/native | 6.x | Navigation core | MIT (FREE) | | @react-navigation/bottom-tabs | 6.x | Bottom tab navigator | MIT (FREE) | | @react-navigation/stack | 6.x | Stack navigator | MIT (FREE) | | react-native-maps | 1.18.x | Native Google/Apple Maps | MIT (FREE) | | expo-location | 18.0.x | Foreground + background GPS | MIT (FREE) | | expo-task-manager | 12.0.x | Background task registration | MIT (FREE) | | expo-notifications | 0.29.x | Push notification handler | MIT (FREE) | | expo-haptics | 14.0.x | Vibration/haptic feedback | MIT (FREE) | | expo-secure-store | 14.0.x |

Encrypted key-value storage (native) | MIT (FREE) | | expo-image-picker | 16.0.x | Camera/gallery image selection | MIT (FREE) | | expo-file-system | 18.0.x | Local file read/write | MIT (FREE) | | expo-linear-gradient | 14.0.x | Gradient backgrounds | MIT (FREE) | | expo-blur | 14.0.x | Blur effects | MIT (FREE) | | expo-constants | 17.0.x | App constants + device info | MIT (FREE) | | expo-device | 7.0.x | Device type detection | MIT (FREE) | | expo-status-bar | 2.0.x | Status bar color control | MIT (FREE) | | expo-linking | 56.0.x | Deep links | MIT (FREE) | | zustand | 5.0.x | Lightweight global state management | MIT (FREE) | | @tanstack/react-query | 5.45.x | Async data fetching + caching | MIT (FREE) | | axios | 1.7.x | HTTP client | MIT (FREE) | | date-fns | 3.6.x | Date formatting utilities | MIT (FREE) | | react-native-reanimated | 3.16.x | 60fps animations on UI thread | MIT (FREE) | | react-native-svg | 15.8.x | SVG rendering | MIT (FREE) | | react-native-gesture-handler | 2.20.x | Touch gesture recognition | MIT (FREE) | | react-native-screens | 4.4.x | Native screen containers | MIT (FREE) | | react-native-safe-area-context | 4.12.x | Safe area insets | MIT (FREE) | | react-native-sse | 1.2.x | Server-Sent Events for native | MIT (FREE) | | react-native-web | 0.21.x | RN → DOM bridge for web compat | MIT (FREE) | | @expo/vector-icons | 14.0.x | Ionicons/MaterialIcons | MIT (FREE) | | @react-native-community/slider | 4.5.x | Radius slider for geofences | MIT (FREE) | | TypeScript | 5.3.x | Static types | Apache-2 (FREE) |

Web Frontend (Landing + Dashboards)

| Library | Version | Purpose | License | |---------|---------|---------|---------| | Leaflet.js | 1.9.4 | Interactive maps (CDN) | BSD-2 (FREE) | | CartoDB Tiles | — | Dark Matter / Light / Voyager map tiles | FREE (under limits) | | ESRI World Imagery | — | Satellite map tiles | FREE (attribution req.) | | Google Fonts | — | Inter typeface | OFL (FREE) | | Vanilla JS | ES2022 | No framework — plain HTML/CSS/JS | FREE |

---

4. Infrastructure Components

4.1 Neon PostgreSQL (PAID)

- **Provider:** Neon.tech — serverless PostgreSQL
- **Extensions:** PostGIS 3.x (spatial queries), uuid-ossp (UUID generation)
- **Connection:** Pooled via `@neondatabase/serverless` HTTP driver (works in edge/serverless environments without WebSocket)
- **Plan:** Neon Free Tier → upgrade to Neon Launch ($19/mo) for production scale
- **Usage in code:**

```
const { neon } = require('@neondatabase/serverless')
const sql = neon(process.env.DATABASE_URL)
```

4.2 Cloudflare R2 (PAID)

- **Provider:** Cloudflare R2 — S3-compatible object storage
- **Zero egress fees** (unlike AWS S3)
- **Used for:** User avatars, circle group icons

- **Access:** AWS SDK v3 (`@aws-sdk/client-s3`) with R2 endpoint
- **Presigned URLs:** Generated server-side, client uploads directly to R2 (no backend bandwidth cost)
- **Free tier:** 10 GB storage, 1M Class A ops/month (free for early stage)
- **Config:**

```
const r2 = new S3Client({
  region: 'auto',
  endpoint: `https://${CF_ACCOUNT_ID}.r2.cloudflarestorage.com`,
  credentials: { accessKeyId: R2_ACCESS_KEY, secretAccessKey: R2_SECRET_KEY }
})
```

4.3 Expo Push Notification Service (PAID at scale)

- **Provider:** Expo — exp.host push API
- **Free tier:** Up to 1,000 active push devices
- **Paid:** Expo EAS ($99+/month) for production volume + priority delivery
- **Mechanism:** Backend POSTs to `https://exp.host/--/api/v2/push/send`
- **Authentication:** `EXPO_ACCESS_TOKEN` in Authorization header
- **Token storage:** Each user's `push_token` stored in `users.push_token` column

4.4 Traccar GPS Server (SELF-HOSTED / FREE)

- **Software:** Traccar (open-source GPS telemetry server)
- **Host:** `gravity.trackalways.com/telemetry`
- **Purpose:** Ingest GPS from dedicated hardware trackers (OBD dongles, personal GPS units)
- **Integration:** Traccar sends webhook POST to `/webhooks/traccar/location` on each position update
- **Cost:** Server hosting cost only (free software)

4.5 Caddy 2 Reverse Proxy (FREE)

- **Auto-HTTPS:** Caddy automatically provisions Let's Encrypt TLS certificates
- **Features:** gzip compression, SSE flush passthrough, security headers, API routing
- **Local dev config:** `:3021` → backend `:8002`
- **Production:** `:443` → backend `:8002` with domain routing

4.6 Redis / BullMQ (OPTIONAL)

- **Purpose:** Job queue for background cleanup tasks
- **Fallback:** If `REDIS_URL` env var not set, falls back to `node-cron` (no Redis needed)
- **Managed options:** Upstash Redis (free tier: 10k cmd/day), Redis Cloud free tier

---

5. Database Schema

5.1 Entity Relationship

```
users ──< circle_members >── circles
  │                              │
  │                          safe_zones
  │                              │
  ├──< device_locations       geofence_events >── users
  ├──< user_latest_locations
  └──< geofence_events
```

5.2 Table Definitions

users

```
CREATE TABLE users (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  phone        VARCHAR(20)  UNIQUE NOT NULL,
  name         VARCHAR(100) NOT NULL,
  email        VARCHAR(255) UNIQUE,
  avatar_url   TEXT,
  push_token   TEXT,                          -- Expo push token
  country_code VARCHAR(5)   NOT NULL DEFAULT 'IN',
  password_hash VARCHAR(255) NOT NULL,
  google_id    TEXT UNIQUE,                   -- Google OAuth sub claim
  created_at   TIMESTAMPTZ  DEFAULT NOW(),
  updated_at   TIMESTAMPTZ  DEFAULT NOW()
);
```

circles

```
CREATE TABLE circles (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        VARCHAR(100) NOT NULL,
  icon_url    TEXT,                           -- R2 object URL
  invite_code VARCHAR(12) UNIQUE NOT NULL,    -- 12-char alphanumeric
  created_by  UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);
```

circle_members

```
CREATE TABLE circle_members (
  id        UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  circle_id UUID NOT NULL REFERENCES circles(id) ON DELETE CASCADE,
```

```
  user_id   UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role      VARCHAR(20) NOT NULL DEFAULT 'member'
              CHECK (role IN ('admin', 'member')),
  joined_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(circle_id, user_id)
);
```

`device_locations`

```
CREATE TABLE device_locations (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  geom        GEOMETRY(Point, 4326) NOT NULL,   -- WGS84 lon/lat
  accuracy    FLOAT,                             -- meters
  speed       FLOAT,                             -- m/s
  bearing     FLOAT,                             -- degrees 0-360
  altitude    FLOAT,                             -- meters
  battery_level FLOAT,                           -- 0.0 to 1.0
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes for performance
CREATE INDEX idx_device_locations_user_id   ON device_locations(user_id);
CREATE INDEX idx_device_locations_geom      ON device_locations USING GIST(geom);
CREATE INDEX idx_device_locations_recorded_at ON device_locations(recorded_at DESC);
```

`user_latest_locations`

```
-- Materialized "current position" — updated via INSERT ... ON CONFLICT DO UPDATE
CREATE TABLE user_latest_locations (
  user_id       UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  geom          GEOMETRY(Point, 4326),
  accuracy      FLOAT,
  battery_level FLOAT,
  updated_at    TIMESTAMPTZ DEFAULT NOW()
);
```

`safe_zones` (Geofences)

```
CREATE TABLE safe_zones (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  circle_id   UUID NOT NULL REFERENCES circles(id) ON DELETE CASCADE,
  name        VARCHAR(100) NOT NULL,
  geom        GEOMETRY(Polygon, 4326) NOT NULL,  -- polygon boundary
  radius_meters FLOAT,                           -- for display radius
  created_by  UUID NOT NULL REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_safe_zones_circle_id ON safe_zones(circle_id);
CREATE INDEX idx_safe_zones_geom      ON safe_zones USING GIST(geom);
```

`geofence_events`

```
CREATE TABLE geofence_events (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  safe_zone_id UUID NOT NULL REFERENCES safe_zones(id) ON DELETE CASCADE,
  event_type   VARCHAR(10) NOT NULL CHECK (event_type IN ('entry', 'exit')),
  geom         GEOMETRY(Point, 4326),              -- position at time of event
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_geofence_events_user_id    ON geofence_events(user_id);
CREATE INDEX idx_geofence_events_created_at ON geofence_events(created_at DESC);
```

5.3 Spatial Query Pattern (PostGIS)

Geofence containment check:

```
SELECT sz.id, sz.name
FROM safe_zones sz
JOIN circle_members cm ON cm.circle_id = sz.circle_id
WHERE cm.user_id = $1
  AND ST_Contains(sz.geom, ST_SetSRID(ST_MakePoint($2, $3), 4326))
```

- `$1` = user UUID
- `$2` = longitude (float)
- `$3` = latitude (float)
- `ST_Contains` returns true if the point falls inside the polygon boundary

---

6. Backend API Reference

Base URL: `https://gravity.trackalways.com/api/v1`
Auth: Bearer JWT in `Authorization` header (all protected routes)
Rate Limit: 100 req/15 min per IP (via express-rate-limit)

6.1 Authentication — `/api/v1/auth`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | POST | `/auth/register` | None | `{phone, name, password, country_code?}` | `{token, user}` | | POST | `/auth/login` | None | `{phone, password}` | `{token, user}` | | POST | `/auth/google` | None | `{id_token}` | `{token, user}` |

Registration flow:

1. Validate with Zod schema
2. `bcrypt.hash(password, 10)` → store `password_hash`
3. `jwt.sign({userId}, JWT_SECRET, {expiresIn: '30d'})`
4. Return JWT + user object

Google OAuth flow:

1. Client receives Google `id_token` via Google Identity Services (GIS)
2. Backend decodes JWT payload (base64url decode, no external library):

```
const payload = JSON.parse(Buffer.from(idToken.split('.')[1], 'base64url').toString())
// payload.sub = Google user ID, payload.email, payload.name
```

3. Upsert user by `google_id` — create if new, return existing if returning user
4. Return Gravity JWT

6.2 Users — `/api/v1/users`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | GET | `/users/me` | JWT | — | `{user}` | | PATCH | `/users/me` | JWT | `{name?, email?, push_token?, avatar_url?}` | `{user}` |

`push_token` **update:** Called by mobile app after Expo push registration — stores device push token for notification delivery.

6.3 Circles — `/api/v1/circles`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | GET | `/circles` | JWT | — | `[{circle, members[]}]` | | POST | `/circles` | JWT | `{name, icon_url?}` | `{circle}` | | GET | `/circles/:id` | JWT | — | `{circle, members[]}` | | POST | `/circles/:id/join` | JWT | `{invite_code}` | `{circle}` | | DELETE | `/circles/:id/leave` | JWT | — | `{success}` | | GET | `/circles/:id/locations` | JWT | — | `[{user_id, lat, lng, battery, updated_at}]` |

Auto invite_code generation:

```
const invite_code = Math.random().toString(36).slice(2, 14).toUpperCase()
```

6.4 Geofences — `/api/v1/geofences`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | GET | `/geofences` | JWT | — | `[{safe_zone}]` | | POST | `/geofences` | JWT | `{circle_id, name, center_lat, center_lng, radius_meters}` | `{safe_zone}` | | DELETE | `/geofences/:id` | JWT | — | `{success}` |

Polygon construction from center + radius: Backend converts `(center_lat, center_lng, radius_meters)` → PostGIS polygon using `ST_Buffer(ST_MakePoint(lng, lat)::geography, radius)::geometry`.

6.5 SOS — `/api/v1/sos`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | POST | `/sos` | JWT | `{latitude?, longitude?, message?}` | `{success, notified_count}` | | GET | `/sos/history` | JWT | — | `[{sos_event}]` |

SOS trigger flow:

1. Authenticated user POSTs `/sos`
2. Backend looks up all circles the user belongs to → all member user IDs
3. Broadcasts SSE event `type: 'sos'` to all connected circle members
4. Sends Expo push notification to all members with a push token

6.6 Media — `/api/v1/media`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | POST | `/media/presigned-url` | JWT | `{filename, content_type}` | `{upload_url, public_url}` |

R2 presigned upload flow:

1. Client requests presigned PUT URL
2. Backend generates 15-min expiry presigned URL via AWS SDK
3. Client PUTs file directly to R2 (no backend bandwidth)
4. Client stores returned `public_url` in their profile

6.7 SSE — `/api/v1/sse`

| Method | Path | Auth | Body | Response | |--------|------|------|------|---------| | GET | `/sse/stream` | JWT (header or `?token=`) | — | `text/event-stream` |

EventSource limitation workaround: Browser's native `EventSource` API cannot send custom headers. The SSE route accepts the JWT via `?token=` query parameter as a fallback (in addition to `Authorization: Bearer` header for native clients).

Event types pushed over SSE:

- `location_update` — a family member's GPS position changed
- `geofence_entry` — a member entered a safe zone
- `geofence_exit` — a member exited a safe zone
- `sos` — emergency SOS triggered by a member
- `heartbeat` — keepalive comment every 30s to prevent connection timeout

6.8 Traccar Webhook — `/webhooks/traccar/location`

| Method | Path | Auth | Body | |--------|------|------|------| | POST | `/webhooks/traccar/location` | Shared secret header | `{deviceId, lat, lng, speed, bearing, battery}` |

Flow:

1. Traccar sends position update

2. Backend maps `deviceId` → Gravity `user_id`
3. Inserts into `device_locations`
4. Upserts `user_latest_locations`
5. Runs geofence check via `geofence.js`
6. Broadcasts SSE `location_update` to all circle members

---

7. Real-Time System (SSE)

Architecture

Gravity uses **Server-Sent Events (SSE)** rather than WebSockets for real-time updates. This was chosen because:

- SSE is unidirectional (server → client), which matches all use cases (location updates, alerts)
- SSE reconnects automatically on disconnect
- SSE works through HTTP/1.1 and most proxies (WebSocket requires protocol upgrade)
- Lower overhead than WebSocket for read-mostly streams

In-Memory Client Registry

```
// services/sse.js
const clients = new Map()   // userId → [res, res, ...]  (multi-tab)

function addClient(userId, res) { ... }
function removeClient(userId, res) { ... }
function sendToUser(userId, eventType, data) {
  const conns = clients.get(userId) || []
  conns.forEach(res => res.write(`event: ${eventType}\ndata: ${JSON.stringify(data)}\n\n`))
}
function sendToUsers(userIds, eventType, data) {
  userIds.forEach(id => sendToUser(id, eventType, data))
}
```

Note: The in-memory registry means SSE connections are node-process local. For multi-instance deployments, a Redis pub/sub adapter would be needed (BullMQ's `events` channel or a dedicated `ioredis` subscriber).

SSE Connection Headers

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
X-Accel-Buffering: no         ← disables Nginx/Caddy buffering
```

Heartbeat

```
const heartbeat = setInterval(() => res.write(': heartbeat\n\n'), 30000)
res.on('close', () => { clearInterval(heartbeat); removeClient(userId, res) })
```

---

8. Mobile Application

8.1 App Structure

```
mobile/
├── index.js                    # Entry point — registerRootComponent
├── app/
│   ├── _layout.jsx             # Root layout + navigation stack
│   └── index.jsx               # Auth guard redirect
└── src/
    ├── screens/
    │   ├── auth/               # Login, Register screens
    │   ├── HomeScreen.jsx      # Live map + family locations
    │   ├── CirclesScreen.jsx   # Family circles management
    │   ├── AlertsScreen.jsx    # Notification history
    │   ├── SafeZonesScreen.jsx # Geofence CRUD
    │   └── ProfileScreen.jsx   # Account settings
    ├── store/
    │   ├── authStore.js        # Zustand — auth state + JWT
    │   └── circleStore.js      # Zustand — circles + members
    ├── services/
    │   ├── api.js              # Axios instance + interceptors
    │   ├── location.js         # Background GPS tracking
    │   └── notifications.js   # Expo push registration
    ├── components/
    │   ├── MemberAvatar.jsx    # Avatar with online indicator
    │   ├── PulseRing.jsx       # Animated location pulse
    │   ├── BatteryIndicator.jsx # Battery level display
    │   └── ui/PremiumButton.jsx # Haptics-enabled button
    ├── hooks/                  # Custom React hooks
    ├── navigation/             # Navigator configuration
    ├── theme/
    │   ├── colors.js           # Dark green palette (#0A5C35)
    │   └── typography.js       # Inter font scale
    └── utils/
        └── storage.js          # Platform-safe storage wrapper
```

8.2 State Management (Zustand)

authStore — global auth state:

```
{
  user: null | { id, name, phone, email, avatar_url },
  token: null | string,
  isLoading: boolean,
  login: (phone, password) => Promise,
  logout: () => void,
  loadFromStorage: () => Promise
}
```

Token persisted to `expo-secure-store` (native) or `localStorage` (web) via `utils/storage.js`.

circleStore — circles data:

```
{
  circles: [],
  activeCircle: null,
  members: [],
  setActiveCircle: (circle) => void,
  fetchCircles: () => Promise
}
```

8.3 API Service

```
// src/services/api.js
const apiClient = axios.create({ baseURL: API_BASE_URL })

apiClient.interceptors.request.use(config => {
  const token = authStore.getState().token
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

TanStack Query wraps all API calls for caching, background refetch, and loading states.

8.4 Background Location Tracking

Platform guard pattern:

```
// src/services/location.js
if (Platform.OS !== 'web') {
  const TaskManager = require('expo-task-manager')
  TaskManager.defineTask(LOCATION_TASK, ({ data, error }) => {
    if (error) return
    const { locations } = data
    // POST to /api/v1/locations with JWT
  })
}
```

Foreground tracking (web fallback):

```
if (Platform.OS === 'web' && navigator.geolocation) {
  navigator.geolocation.watchPosition(pos => postLocation(pos.coords), ...)
}
```

Location update payload:

```
{
  "latitude": 28.6139,
  "longitude": 77.2090,
  "accuracy": 12.5,
  "speed": 0.0,
  "bearing": 0.0,
  "altitude": 216.0,
  "battery_level": 0.85
}
```

8.5 Push Notifications (Mobile)

```
// src/services/notifications.js
async function registerForPushNotifications() {
  if (Platform.OS === 'web') return null

  const Notifications = require('expo-notifications')
  const { status } = await Notifications.requestPermissionsAsync()
  if (status !== 'granted') return null

  const token = await Notifications.getExpoPushTokenAsync({
    projectId: Constants.expoConfig.extra.eas.projectId
  })

  // Store token on backend
  await api.patch('/users/me', { push_token: token.data })
  return token.data
}
```

8.6 Web Compatibility Layer

All native-only APIs are guarded to allow web builds:

API	Native	Web Fallback
`expo-secure-store`	SecureStore.getItemAsync	`localStorage`
`expo-haptics`	Haptics.impactAsync	(no-op)
`expo-task-manager`	TaskManager.defineTask	not registered
`expo-notifications`	full push support	returns null
`react-native-sse`	NativeEventSource	browser EventSource
`react-native-maps`	MapView native	stubs/react-native-maps.web.js

---

9. Web Platform

9.1 Landing Page (`index.html`)

Features:

- Interactive Leaflet.js map with animated family member markers
- Particle canvas background animation
- Stats counter (animated: users, countries, alerts)
- Map type switcher: Dark Matter / Light / Satellite / Street
- Features grid, testimonials, download CTA
- Google Play Store link: `https://play.google.com/store/apps/details?id=com.trackalways.gravity`
- Responsive mobile layout

Map tile sources: | Mode | Provider | URL Template | |------|----------|-------------| | Dark | CartoDB | `https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png` | | Light | CartoDB | `https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png` | | Satellite | ESRI | `https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/...` | | Street | CartoDB | `https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png` |

9.2 Parent Dashboard (`parent-panel.html`)

5-tab UI:

| Tab | Content | |-----|---------| | Map | Leaflet map with live family member positions, custom markers with avatar + battery | | Family | Member list with last seen, location, battery indicators | | Alerts | Real-time alert feed (geofence events, SOS) | | Geofence | Safe zone list + create/delete controls | | Settings | Account info, notifications toggle, logout |

Auth guard:

```
const token = localStorage.getItem('gravity_token')
if (!token) {
  localStorage.setItem('gravity_redirect', 'parent-panel.html')
  window.location.href = 'login.html'
}
```

Real-time SSE connection:

```
const es = new EventSource(`${API_BASE}/sse/stream?token=${token}`)
es.addEventListener('location_update', e => updateMemberOnMap(JSON.parse(e.data)))
es.addEventListener('geofence_entry', e => showAlert(JSON.parse(e.data)))
es.addEventListener('geofence_exit', e => showAlert(JSON.parse(e.data)))
es.addEventListener('sos', e => triggerSOSAlert(JSON.parse(e.data)))
```

9.3 Child Dashboard (`child-panel.html`)

5-tab UI:

Tab	Content
Home	Welcome, status card, quick actions
Map	Child's own location on Leaflet map
SOS	Large SOS button with 3-second hold confirmation
Family	View circle members + their locations
Profile	Account info, logout

SOS button — hold-to-confirm UX:

```
let holdTimer = null
let holdCount = 3

button.addEventListener('mousedown', () => {
  holdTimer = setInterval(() => {
    holdCount--
    updateButtonLabel(holdCount)
    if (holdCount === 0) { clearInterval(holdTimer); activateSOS() }
  }, 1000)
})
button.addEventListener('mouseup', () => { clearInterval(holdTimer); resetButton() })
```

SOS with geolocation:

```
function activateSOS() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(
      pos => sendSOS(pos.coords.latitude, pos.coords.longitude),
      () => sendSOS(null, null)
    )
  }
}
```

9.4 Login Page (`login.html`)

Tabs: Login | Register

Phone + Password auth:

```
const res = await fetch(`${API_BASE}/auth/login`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ phone, password })
})
const { token, user } = await res.json()
localStorage.setItem('gravity_token', token)
localStorage.setItem('gravity_user', JSON.stringify(user))
```

Google Identity Services:

```
google.accounts.id.initialize({
  client_id: GOOGLE_CLIENT_ID,
  callback: async ({ credential }) => {
    const res = await fetch(`${API_BASE}/auth/google`, {
      method: 'POST',
      body: JSON.stringify({ id_token: credential })
    })
  }
})
```

9.5 Marketing Pages

- `parent.html` — Parent features marketing page with feature cards, screenshots, CTA
- `child.html` — Child features marketing page with safety messaging, CTA

---

10. Authentication & Security

10.1 JWT Implementation

| Property | Value | |----------|-------| | Algorithm | HS256 | | Secret | `JWT_SECRET` env var (min 32 chars recommended) | | Expiry | 30 days (`30d`) | | Payload | `{ userId: UUID, iat, exp }` | | Storage (native) | `expo-secure-store` (encrypted, OS keychain) | | Storage (web) | `localStorage` (acceptable for SPA, note XSS risk) |

Auth middleware:

```
function authenticate(req, res, next) {
  const header = req.headers.authorization
  if (!header?.startsWith('Bearer ')) return res.status(401).json({ error: 'Unauthorized' })
  const token = header.slice(7)
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET)
    next()
  } catch {
    res.status(401).json({ error: 'Invalid token' })
  }
}
```

10.2 Password Security

- Algorithm: bcrypt (via `bcryptjs`)
- Cost factor: 10 rounds (~100ms per hash)
- Storage: `password_hash` column only — plain password never persisted

10.3 HTTP Security Headers (Helmet)

```
app.use(helmet())   // sets:
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
// X-XSS-Protection: 1; mode=block
// Strict-Transport-Security: max-age=15552000
// Content-Security-Policy: default-src 'self'
```

10.4 Rate Limiting

```
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 100,                   // 100 requests per window
  standardHeaders: true,
  legacyHeaders: false
})
app.use('/api/', limiter)
```

10.5 Input Validation (Zod)

All POST request bodies are validated with Zod schemas before processing:

```
const registerSchema = z.object({
  phone: z.string().min(10).max(15),
  name: z.string().min(2).max(100),
  password: z.string().min(8),
  country_code: z.string().length(2).optional()
})
```

10.6 Circle Authorization

All circle operations verify membership:

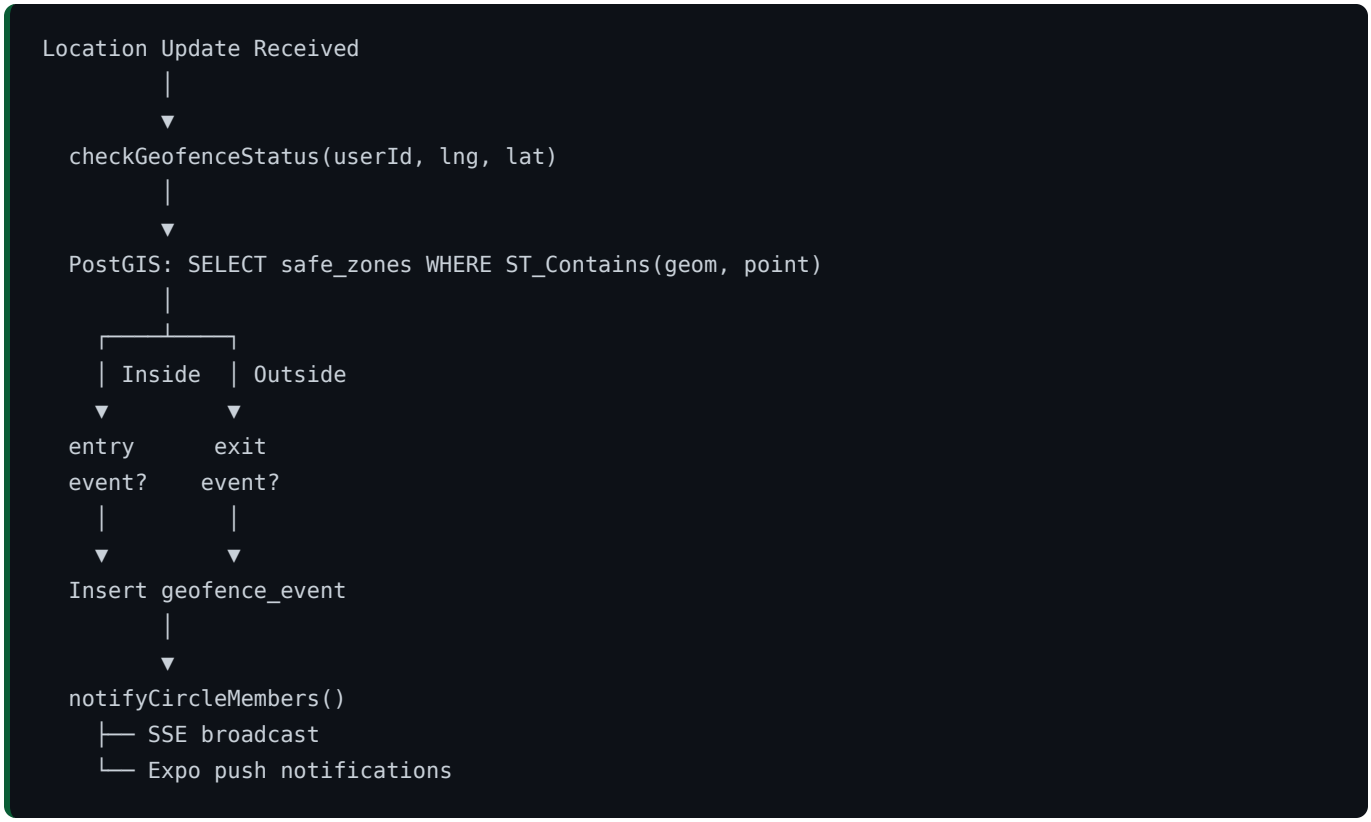
```
SELECT 1 FROM circle_members
WHERE circle_id = $1 AND user_id = $2
```

Only circle admins can delete geofences and remove members.

---

11. Geofencing Engine

11.1 Flow



11.2 Entry/Exit Detection

The service tracks the previous state to avoid duplicate events:

```
const prevState = await getPreviousGeofenceState(userId, safeZoneId)
if (isInside && prevState !== 'inside') {
  await insertGeofenceEvent(userId, safeZoneId, 'entry', geom)
  await notifyCircleMembers(userId, safeZone, 'entry')
} else if (!isInside && prevState === 'inside') {
  await insertGeofenceEvent(userId, safeZoneId, 'exit', geom)
  await notifyCircleMembers(userId, safeZone, 'exit')
}
```

11.3 Push Notification Payload

```
{
  to: user.push_token,
  sound: 'default',
  title: `${memberName} ${eventType === 'entry' ? 'arrived at' : 'left'} ${zoneName}`,
  body: `${new Date().toLocaleTimeString()}`,
```

```
  data: { type: 'geofence', event_type: eventType, zone_name: zoneName }
}
```

---

12. Background Jobs

12.1 Job Runner Architecture

```
// jobs/index.js
if (process.env.REDIS_URL) {
  // BullMQ with Redis persistence
  const queue = new Queue('cleanup', { connection: redisClient })
  const worker = new Worker('cleanup', async job => { ... }, { connection: redisClient })
} else {
  // Fallback: node-cron (no Redis needed)
  cron.schedule('0 2 * * *', cleanOldLocations)   // 2:00 UTC daily
  cron.schedule('0 3 * * *', cleanOldGeofenceEvents) // 3:00 UTC daily
}
```

12.2 Scheduled Tasks

| Task | Schedule | Query | |------|----------|-------| | Clean old locations | Daily 02:00 UTC | `DELETE FROM device_locations WHERE recorded_at < NOW() - INTERVAL '7 days'` | | Clean old geofence events | Daily 03:00 UTC | `DELETE FROM geofence_events WHERE created_at < NOW() - INTERVAL '30 days'` |

12.3 Data Retention Policy

| Data Type | Retention | |-----------|-----------| | Location history | 7 days | | Geofence events | 30 days | | SOS events | Indefinite (safety records) | | User accounts | Until deletion request | | Latest location | Always (1 row per user) |

---

13. Push Notification Pipeline

13.1 Expo Push API

```
Backend → POST https://exp.host/--/api/v2/push/send
Authorization: Bearer ${EXPO_ACCESS_TOKEN}
Content-Type: application/json

{
  "to": "ExponentPushToken[xxxxx]",
  "sound": "default",
  "title": "Sarah left School",
```

```
  "body": "3:45 PM",
  "data": { "type": "geofence_exit" }
}
```

13.2 Token Registration Flow

1. App starts → `registerForPushNotifications()`
2. OS prompts user for notification permission
3. If granted → `Notifications.getExpoPushTokenAsync()` → `ExponentPushToken[...]`
4. Token POST to `PATCH /api/v1/users/me` → stored in `users.push_token`
5. Backend uses token for all subsequent push deliveries

13.3 Notification Types

| Event | Title | Body | |-------|-------|------| | SOS | 🚨 `{name} needs help!` | Location coordinates if available | | Geofence entry | `{name} arrived at {zone}` | Timestamp | | Geofence exit | `{name} left {zone}` | Timestamp | | Low battery | `{name}'s battery is low` | Battery percentage |

---

14. Media Storage (R2)

14.1 Upload Flow

14.2 Presigned URL Generation

```
const command = new PutObjectCommand({
  Bucket: process.env.R2_BUCKET_NAME,
  Key: `avatars/${userId}/${filename}`,
  ContentType: contentType
})
```

```
const uploadUrl = await getSignedUrl(r2Client, command, { expiresIn: 900 })
const publicUrl = `https://${process.env.R2_PUBLIC_DOMAIN}/avatars/${userId}/${filename}`
```

---

15. GPS Telemetry (Traccar)

15.1 Integration Overview

Traccar is an open-source GPS tracking platform that supports 170+ GPS device protocols. Gravity integrates with Traccar for hardware GPS trackers (OBD port devices, personal GPS beacons).

15.2 Webhook Flow

```
GPS Device → Traccar Server → Webhook POST → Gravity Backend
```

Webhook endpoint: `POST /webhooks/traccar/location`

Payload from Traccar:

```
{
  "deviceId": "tracker_abc123",
  "latitude": 28.6139,
  "longitude": 77.2090,
  "speed": 12.5,
  "bearing": 245.0,
  "attributes": {
    "batteryLevel": 0.78
  }
}
```

Backend processing:

1. Validate shared-secret header
2. Look up `deviceId` → `user_id` mapping (stored in `users.device_id` or separate mapping table)
3. Insert location record
4. Update `user_latest_locations`
5. Run geofence check
6. Broadcast SSE to circle members

---

16. Free vs Paid Services Breakdown

16.1 Fully Free Components

| Service | Why Free | Limits | |---------|----------|--------| | Node.js 20 | Open source runtime | None | | Express 5.2 | Open source framework | None | | All npm packages | MIT/Apache open source | None | | Expo (toolchain) | Free tier available | OTA update limits | | React Native | Open source | None | | Leaflet.js | BSD-2 open source | None | | CartoDB map tiles | Free for low-traffic sites | Rate limits apply | | ESRI World Imagery | Free with attribution | Rate limits apply | | Traccar | Open source, self-hosted | Hosting cost only | | Caddy 2 | Open source + auto-TLS | None | | Let's Encrypt TLS | Free CA | 50 certs/domain/week | | PostGIS | Open source extension | None | | uuid-ossp | Open source | None | | Google Fonts (Inter) | Free | None | | Google Identity Services | Free OAuth | 10,000 users/day free | | node-cron | Open source | None |

16.2 Paid / Freemium Components

| Service | Free Tier | Paid Plan | When to Upgrade | |---------|-----------|-----------|----------------| | **Neon PostgreSQL** | 0.5 GB storage, 1 compute unit, 1 project | Launch: $19/mo (10 GB, 4 CU) | >500 active users | | **Cloudflare R2** | 10 GB/mo storage, 1M Class A ops | $0.015/GB beyond free | >500 avatars stored | | **Expo Push (EAS)** | 1,000 devices, 10 push/day limit | EAS Production: $99/mo | >1,000 app users | | **Redis (BullMQ)** | Upstash free: 10k cmd/day | Upstash Pay-per-use ~$0.2/100k cmd | Multi-instance deploy | | **VPS / Server** | — | ~$6-20/mo (DigitalOcean, Hetzner) | Always (required for backend) | | **Domain** | — | ~$12/year | Always (required) |

16.3 Monthly Cost Estimate

| Stage | Users | Est. Cost/Month | |-------|-------|----------------| | Development | 0-10 | ~$6-20 (VPS only) | | Early beta | 10-500 | ~$25-40 (VPS + Neon Launch) | | Growth | 500-5000 | ~$130-200 (+ Expo EAS) | | Scale | 5000+ | Custom pricing |

---

17. Feature Matrix (Free vs Premium)

Business model: freemium app — basic features free, premium features via in-app subscription.

17.1 App Features

| Feature | Free | Premium | |---------|:----:|:-------:| | Join/create 1 family circle | ✓ | ✓ | | Create up to 3 family circles | — | ✓ | | See family member locations (real-time) | ✓ | ✓ | | Location history (last 24 hours) | ✓ | — | | Location history (7 days) | — | ✓ | | Up to 3 geofences | ✓ | ✓ | | Unlimited geofences | — | ✓ | | Geofence entry/exit alerts | ✓ | ✓ | | SOS emergency alert | ✓ | ✓ | | SOS with GPS coordinates | ✓ | ✓ | | Push notifications | ✓ | ✓ | | Battery level monitoring | ✓ | ✓ | | Custom avatar + circle icon | — | ✓ | | Speed alerts | — | ✓ | | Driving reports | — | ✓ | | Route history playback | — | ✓ | | Check-in reminders | — | ✓ | | Priority push delivery | — | ✓ | | Hardware GPS tracker support (Traccar) | — | ✓ | | Up to 5 members per circle | ✓ | — | | Unlimited members per circle | — | ✓ | | Ad-free experience | — | ✓ |

17.2 Web Dashboard Features

| Feature | Free | Premium | |---------|:----:|:-------:| | Parent web panel access | ✓ | ✓ | | Child web panel access | ✓ | ✓ | | Live map view | ✓ | ✓ | | Map type switcher (4 styles) | ✓ | ✓ | | Alert history (last 48 hours) | ✓ | — | | Full alert history | — | ✓ | | Geofence management UI | ✓ | ✓ | | Export location data (CSV) | — | ✓ |

---

18. Environment Configuration

18.1 Backend `.env`

```
# Server
PORT=8002
NODE_ENV=production

# Database (Neon PostgreSQL)
DATABASE_URL=postgresql://user:pass@ep-xxx.us-east-2.aws.neon.tech/gravity?sslmode=require

# JWT
JWT_SECRET=your-super-secret-jwt-key-min-32-chars

# Cloudflare R2
R2_ACCOUNT_ID=your_cf_account_id
R2_ACCESS_KEY=your_r2_access_key
R2_SECRET_KEY=your_r2_secret_key
R2_BUCKET_NAME=gravity-media
R2_PUBLIC_DOMAIN=pub-xxxx.r2.dev

# Expo Push Notifications
EXPO_ACCESS_TOKEN=your_expo_access_token

# Redis (optional — BullMQ job queue)
REDIS_URL=redis://localhost:6379

# Traccar webhook secret
TRACCAR_WEBHOOK_SECRET=your_traccar_secret

# CORS
ALLOWED_ORIGINS=https://gravity.trackalways.com,http://localhost:3021
```

18.2 Mobile `app.config.js` (key fields)

```
{
  name: "Gravity",
  slug: "gravity",
  version: "1.0.0",
  platforms: ["ios", "android", "web"],
```

```
  android: {
    package: "com.trackalways.gravity",
    permissions: ["ACCESS_FINE_LOCATION", "ACCESS_BACKGROUND_LOCATION"]
  },
  ios: {
    bundleIdentifier: "com.trackalways.gravity",
    infoPlist: {
      NSLocationAlwaysAndWhenInUseUsageDescription: "Gravity tracks location for family safety",
      NSLocationWhenInUseUsageDescription: "Gravity uses location to show your family where you a
re"
    }
  },
  extra: {
    eas: { projectId: "your-eas-project-id" },
    apiUrl: "https://gravity.trackalways.com/api/v1"
  }
}
```

19. Deployment & Reverse Proxy

19.1 Production Deployment (Caddy)

```
gravity.trackalways.com {
  # SSE — disable buffering for real-time events
  @sse path /api/v1/sse/*
  handle @sse {
    reverse_proxy localhost:8002 {
      flush_interval -1
      header_up X-Forwarded-For {remote_host}
    }
  }

  # API
  handle /api/* {
    reverse_proxy localhost:8002 {
      header_up X-Forwarded-For {remote_host}
      header_up X-Real-IP {remote_host}
    }
  }

  # Webhooks
  handle /webhooks/* {
    reverse_proxy localhost:8002
  }

  # Static web files
  handle {
    root * /media/server/linux-part/Gravity/landing
    file_server
```

```
  }

  # Security headers
  header {
    X-Content-Type-Options nosniff
    X-Frame-Options DENY
    Referrer-Policy strict-origin-when-cross-origin
  }

  encode gzip
}
```

19.2 Process Management (PM2)

```
pm2 start src/app.js --name gravity-backend --watch
pm2 startup
pm2 save
```

19.3 Database Migrations

```
cd backend
node src/db/migrate.js    # runs all .sql files in migrations/
```

Migration files:

- `001_initial.sql` — full schema creation
- `004_add_google_id.sql` — adds `google_id` column to users

---

20. Security Implementation

20.1 Defense in Depth

| Layer | Control | Implementation | |-------|---------|---------------| | Transport | TLS 1.3 | Caddy + Let's Encrypt | | HTTP | Security headers | Helmet.js | | Network | Rate limiting | express-rate-limit (100/15min) | | Auth | JWT verification | jsonwebtoken (HS256) | | Password | Bcrypt hashing | bcryptjs (10 rounds) | | Input | Schema validation | Zod | | DB | Parameterized queries | pg prepared statements | | Authorization | Circle membership check | SQL before every circle op | | Storage (native) | Encrypted keychain | expo-secure-store | | Media | Presigned URLs | 15-min expiry, no proxy needed | | Webhook | Shared secret | X-Traccar-Secret header |

20.2 SQL Injection Prevention

All database queries use parameterized statements — no string interpolation:

```
// SAFE — parameterized
const result = await pool.query(
  'SELECT * FROM users WHERE phone = $1',
  [req.body.phone]
)

// NEVER done — string concat
// `SELECT * FROM users WHERE phone = '${req.body.phone}'`  ← NEVER
```

20.3 CORS Configuration

```
const allowedOrigins = process.env.ALLOWED_ORIGINS.split(',')
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) callback(null, true)
    else callback(new Error('Not allowed by CORS'))
  },
  credentials: true
}))
```

20.4 Known Security Considerations

| Item | Status | Notes | |------|--------|-------| | JWT in localStorage (web) | Acceptable | XSS risk mitigated by CSP headers | | SSE token in query param | Accepted tradeoff | EventSource API limitation — token in URL may appear in server logs; use HTTPS always | | Google JWT decoded without verification | ⚠ Improve | Should verify signature using Google's public keys for production | | No refresh token | MVP | Add refresh tokens for production to reduce 30-day token lifetime | | No 2FA | MVP | Consider TOTP for admin accounts |

End of Engineering Specification — Trackalways Gravity v1.0.0

Document generated: June 2026